AGENTIC AI / MULTI-AGENT SYSTEMS

Group Assignment

LaunchMind

Build a Startup Using a Team of Collaborating AI Agents

Group size: 2–3 students (should be same as project)

Deadline: 12th April 2026

LLM APIs: Any (Claude, OpenAI, Gemini, Mistral, or mixed)

1. What Is This Assignment?

This is a group assignment worth the more weight as compared to other assignments. You will build a real Multi-Agent System (MAS) that autonomously runs a micro-startup from receiving an idea all the way to writing code, posting on Slack, opening GitHub pull requests, and sending emails without a human doing any of it manually.

The key word is real. Your agents will not just print things to a terminal. They will take actual actions on actual platforms:

- A real GitHub repository with real commits and a real pull request opened by your Engineer agent
- A real Slack workspace where your agents post real messages
- A real email sent to a real inbox via SendGrid or the Gmail API

This is not a chatbot. It is not a pipeline where you call one function, get a response, then manually pass it to the next. It is a network of autonomous, LLM-powered agents that reason independently, communicate with each other using structured messages, and take actions in the world based on what other agents tell them.

> **Why this assignment?**
> Almost no undergraduate or masters student at our campus has built this. A working MAS with real platform integrations, agent-to-agent communication, and LLM-powered reasoning is exactly what AI, software engineering, and product roles look for in 2025-26. When you present this in an interview, you will not say 'I built a chatbot.' You will say: 'I built a system where autonomous agents collaborated across Slack, GitHub, and email to run a startup.'

2. Your Startup Idea

Your group picks any startup idea you want. There are no restrictions on domain. The idea should be concrete enough that it has a product something that can be described, partially built, and marketed.

Some random examples of startup ideas:

- A mobile app that connects students with local tutors
- A browser extension that summarises long articles before you read them
- A tool that helps freelancers track unpaid invoices and auto-send reminders
- A platform where restaurants list end-of-day unsold food at a discount
- A CLI tool that auto-generates test cases from Python docstrings

What makes an idea work for this assignment: it needs to have a target user, a core feature, and a reason someone would pay for it. Your agents will need to reason about these things so the vaguer your idea, the weaker your agent outputs will be.

> *You do not need to build a real working product. The Engineer agent will write a landing page or a small prototype not the full product. What matters is that the agents collaborate to define, build, and market it as if they were a real team.*

3. The Agents Who They Are and What They Do

Your system must contain at least five agents. Each agent has a specific role, a specific set of inputs it receives, and a specific set of outputs it produces. Each student in the group owns at least one agent end-to-end meaning you write it, you test it, and you are responsible for it working in the final demo.

Below is a complete description of each agent. Read this carefully this is your specification.

Agent 1: CEO Agent (Orchestrator)

The CEO agent is the brain of the system. It receives the startup idea as input and is responsible for everything that happens after that.

What it does:

- Receives the startup idea as a plain text description (e.g. 'A platform where students list second-hand textbooks for sale')
- Breaks the idea into structured tasks one for the Product agent, one for the Engineer agent, one for the Marketing agent
- Sends each task as a structured JSON message to the correct agent
- Waits for responses from each agent
- Reviews those responses using LLM reasoning it does not just forward them, it actually thinks about whether the output is good enough
- If an agent's output is poor or incomplete, it sends a follow-up instruction asking for revision
- Once all agents have produced acceptable output, it compiles a final summary and sends it to the Slack channel

What it must NOT do:

- It must not hardcode the task list. The tasks must be generated by the LLM based on the idea.
- It must not skip the review step. Simply passing Agent A's output to Agent B without reading it is not orchestration it is piping.

Inputs:

- Startup idea (string, provided by you at runtime)

Outputs:

- Structured JSON task messages sent to each sub-agent
- A final summary Slack message posted to the team channel
- A log of every decision it made and why

LLM prompt requirements:

The CEO agent must use an LLM at least twice during its lifecycle:

1. Once to decompose the startup idea into tasks for each agent
2. Once to review each agent's output and decide whether it is acceptable or needs revision

> *This agent is the hardest to build well. It requires the most LLM reasoning. Assign it to the most technically confident member of your group.*

Agent 2: Product Agent

The Product agent thinks like a product manager. It takes the task sent by the CEO and produces a structured product specification.

What it does:

- Receives a task message from the CEO agent containing the startup idea and any specific focus areas
- Uses an LLM to generate a full product specification including: a one-sentence value proposition, two or three user personas with names and specific pain points, a list of five core features ranked by priority, and three user stories in the format 'As a [user], I want to [action] so that [benefit]'
- Sends the product specification as a structured JSON object to both the Engineer agent and the Marketing agent
- Sends a short confirmation message back to the CEO agent indicating the spec is ready

Inputs:

- Task message from CEO agent (JSON)

Outputs:

- Product specification JSON sent to Engineer and Marketing agents
- Confirmation message sent back to CEO agent

What the product spec JSON must contain:

Field	Type	Description
value_proposition	string	One sentence describing what the product does and for whom
personas	array	Each persona has: name, role, and pain_point
features	array	Each feature has: name, description, and priority (1=highest)
user_stories	array	Three stories in standard 'As a / I want / So that' format

Agent 3: Engineer Agent

The Engineer agent is the builder. It reads the product spec and produces real, runnable code and it takes real action on GitHub.

What it does:

- Receives the product specification from the Product agent
- Uses an LLM to generate a complete, working HTML landing page for the startup including a headline, subheadline, a features section, a call-to-action button, and basic CSS styling
- Creates a new GitHub issue on your group's public repository with the title 'Initial landing page' and a description generated by the LLM
- Commits the generated HTML file to a new branch on your repository
- Opens a pull request on GitHub with a title and body generated by the LLM
- Sends the PR URL and the GitHub issue URL back to the CEO agent as a structured message

GitHub requirements this is not optional:

- You must create a public GitHub repository before starting the assignment
- The Engineer agent must authenticate using a GitHub Personal Access Token (PAT) stored as an environment variable never hardcoded in the code
- The commit must be authored by the agent (you can set the author name to something like 'EngineerAgent <agent@launchmind.ai>')
- The pull request must be visible in the repository when your evaluator checks it

Inputs:

- Product specification JSON from Product agent

Outputs:

- HTML landing page file committed to GitHub
- GitHub issue (URL returned to CEO)
- GitHub pull request (URL returned to CEO)

> *The GitHub API can be called using the 'requests' library in Python or 'axios' in JavaScript. You do not need PyGitHub or Octokit plain HTTP calls to api.github.com work fine. See Section 6 for code examples.*

Agent 4: Marketing Agent

The Marketing agent thinks like a growth marketer. It reads the product spec and produces real marketing outputs including sending a real email.

What it does:

- Receives the product specification from the Product agent
- Uses an LLM to generate: a product tagline (under 10 words), a short product description for a landing page (2-3 sentences), a cold outreach email addressed to a potential early user or investor, and three social media post drafts (for Twitter/X, LinkedIn, and Instagram)
- Sends the cold outreach email to a designated test email address using SendGrid or the Gmail API
- Posts a Slack message to your group's designated channel summarising the product launch the message must include the tagline, a one-line description, and a link to the GitHub PR
- Sends all generated copy back to the CEO agent as a structured message

Email requirements:

- You must use SendGrid (free tier: 100 emails/day) or the Gmail API with OAuth2
- The 'from' address must be a verified sender in your SendGrid account or your Gmail account
- The email subject line and body must be generated by the LLM not hardcoded
- Send the email to yourself or a test address you control do not send unsolicited emails to real people

Slack requirements:

- You must create a free Slack workspace for this assignment.
- The Marketing agent must post using a Slack Bot Token (stored as environment variable)
- The message must appear in a channel called #launches or similar
- The message must be formatted using Slack Block Kit not plain text

Inputs:

- Product specification JSON from Product agent
- GitHub PR URL from CEO agent (received before posting to Slack)

Outputs:

- Real email sent to test inbox
- Real Slack message posted to workspace
- All copy sent back to CEO agent as structured JSON

Agent 5: QA / Reviewer Agent (Required for groups of 3)

If your group has three members, one of you must build the QA agent. If your group has two members, this agent is optional but earns bonus marks.

What it does:

- Receives the Engineer agent's output (HTML code) and the Marketing agent's copy from the CEO agent
- Uses an LLM to review the HTML landing page: does it match the product spec? Is the headline consistent with the value proposition? Are the features mentioned?
- Uses an LLM to review the marketing copy: is the tagline compelling? Does the cold email have a clear call to action? Is the tone appropriate?
- Posts review comments on the GitHub pull request using the GitHub API at least two inline comments on the HTML file
- Sends a structured review report back to the CEO agent, including a pass/fail verdict and a list of specific issues
- If the verdict is 'fail', the CEO agent must instruct the relevant agent to revise its output this is where dynamic decision-making happens

Inputs:

- Engineer output (HTML + PR URL) from CEO agent
- Marketing output (copy JSON) from CEO agent

Outputs:

- GitHub PR review comments (posted via API)
- Structured review report sent to CEO agent

---

4. How Agents Must Communicate

This is the most important section. Do not skip it.

Agents must communicate by passing structured messages to each other. These messages must follow a defined schema. You cannot just pass raw text between agents you need to know exactly what each agent sent, what it contained, and when it was sent.

4.1 The Message Schema

Every message passed between agents must be a JSON object with the following fields:

Field	Type	Required	Description
message_id	string	Yes	A unique ID for this message (e.g. UUID)
from_agent	string	Yes	Name of the sending agent (e.g. 'ceo', 'product', 'engineer')
to_agent	string	Yes	Name of the receiving agent
message_type	string	Yes	One of: 'task', 'result', 'revision_request', 'confirmation'
payload	object	Yes	The actual content different for each message type
timestamp	string	Yes	ISO 8601 timestamp of when the message was created
parent_message_id	string	No	ID of the message this is responding to (for traceability)

Example of a valid task message from the CEO to the Product agent:

```
{   "message_id": "msg-001",   "from_agent": "ceo",   "to_agent": "product",
"message_type": "task",   "payload": {     "idea": "A platform where students list
second-hand textbooks for sale",     "focus": "Define the core user personas and top 5
features"   },   "timestamp": "2025-10-01T09:00:00Z" }
```

4.2 The Message Bus

You need a way for agents to send messages to each other. There are three acceptable approaches:

Option A: Shared Python dictionary or JSON file (simplest)

All agents run in the same process. You have a shared dictionary called message_bus. When Agent A wants to send a message to Agent B, it appends the message to message_bus['agent_b']. Agent B reads from message_bus['agent_b'] to get its messages. This is the simplest approach and fully acceptable.

Option B: SQLite database (intermediate)

All agents write messages to a messages table in a local SQLite database. Each agent polls for new messages addressed to it. This works well if you want agents to run in separate threads or processes.

Option C: Redis pub/sub (advanced, bonus marks)

Each agent subscribes to a Redis channel named after itself. Messages are published to the recipient's channel. This is how production MAS systems work. It earns bonus marks but is not required.

> *Whichever option you choose, you must be able to show the full message history in your demo. Your evaluator will ask: 'Show me every message the CEO agent sent and received.' You must be able to answer this.*

4.3 The Required Collaboration Moment

Your system must demonstrate at least one instance of dynamic decision-making: the CEO agent must receive a response from another agent, reason about it using an LLM, and change its next action based on that reasoning.

The minimum acceptable version: the CEO agent receives the Product agent's spec, sends it to the LLM with the prompt 'Review this product spec. Is it specific enough? If not, what is missing?', and if the LLM says it is not specific enough, sends a revision_request back to the Product agent with specific feedback.

A stronger version (recommended): the QA agent reviews the Engineer's HTML and returns a fail verdict. The CEO agent receives this, reasons about it, and instructs the Engineer agent to revise the HTML addressing the specific issues raised.

> *If your agents simply run one after another in a fixed sequence with no feedback loops, you will lose 25% of your grade (the collaboration and dynamic decision-making criterion). This is not negotiable.*

---

5. Platform Setup Step by Step

This section tells you exactly how to set up each platform. Follow it before you write a single line of agent code.

5.1 GitHub

3. Create a public GitHub repository named 'launchmind-[your-group-name]'
4. Go to Settings > Developer Settings > Personal Access Tokens > Tokens (classic)
5. Generate a new token with the following permissions: repo (full control), workflow
6. Copy the token and save it you will never see it again after closing the page
7. Store it as an environment variable: export GITHUB_TOKEN='your_token_here'
8. Do not ever put this token in your code or commit it to the repository

9. Create a .env file, add GITHUB_TOKEN=your_token_here, and add .env to your .gitignore
10. Test that your token works by running: curl -H 'Authorization: token $GITHUB_TOKEN' https://api.github.com/user

5.2 Slack

11. Go to api.slack.com/apps and click 'Create New App'
12. Choose 'From scratch', name it 'LaunchMind Bot', and select your workspace
13. In 'OAuth & Permissions', scroll to 'Bot Token Scopes' and add: chat:write, channels:read, channels:join
14. Click 'Install to Workspace' and copy the Bot User OAuth Token (starts with xoxb-)
15. Store as environment variable: export SLACK_BOT_TOKEN='xoxb-your-token'
16. Create a channel in your workspace called #launches
17. Invite the bot to the channel by typing /invite @LaunchMindBot in the channel
18. Test the bot by running a quick Python script that calls the Slack API chat.postMessage endpoint

5.3 SendGrid (Email)

19. Go to sendgrid.com and create a free account (100 emails/day, no credit card required)
20. Go to Settings > API Keys and create a new API key with 'Mail Send' permission
21. Store as environment variable: export SENDGRID_API_KEY='SG.your_key_here'
22. Go to Settings > Sender Authentication and verify a single sender email address (use your own email)
23. This verified email will be the 'from' address in all emails your Marketing agent sends
24. Test by sending a test email using the SendGrid Python library before integrating it into the agent

---

6. Technical Guidance and Code Patterns

This section gives you concrete starting points for the trickiest parts of the implementation.

6.1 Recommended Frameworks

Framework	Best for	Difficulty	Docs
CrewAI	Beginners role-based agents, built-in task assignment	Low	docs.crewai.com

Framework	Best for	Difficulty	Docs
AutoGen (Microsoft)	Agent-to-agent conversation loops	Medium	microsoft.github.io/autogen
LangGraph	Precise control over agent flow and state	Medium-High	langchain-ai.github.io/langgraph
Custom (direct API calls)	Full control, no abstraction	High	platform.openai.com / docs.anthropic.com

Recommendation: if no one in your group has used an agent framework before, start with CrewAI. It maps almost perfectly onto this assignment's structure you define an agent per role and a task per action, and CrewAI handles the communication loop. You can always refactor later.

6.2 Calling the GitHub API

To open a pull request from the Engineer agent:

```
import requests, os  GITHUB_TOKEN = os.environ["GITHUB_TOKEN"] REPO = "your-
username/launchmind-your-group" HEADERS = {     "Authorization": f"token {GITHUB_TOKEN}",
"Accept": "application/vnd.github+json" }  # 1. Create a branch
requests.post(f"https://api.github.com/repos/{REPO}/git/refs",     headers=HEADERS,
json={"ref": "refs/heads/agent-landing-page", "sha": "<base_sha>"})  # 2. Commit a file
import base64 content = base64.b64encode(html_content.encode()).decode()
requests.put(f"https://api.github.com/repos/{REPO}/contents/index.html",
headers=HEADERS,     json={"message": "Add landing page", "content": content,
"branch": "agent-landing-page"})  # 3. Open PR r =
requests.post(f"https://api.github.com/repos/{REPO}/pulls",     headers=HEADERS,
json={"title": "Initial landing page", "body": pr_body,           "head": "agent-landing-
page", "base": "main"}) pr_url = r.json()["html_url"]
```

6.3 Posting to Slack with Block Kit

The Marketing agent must use Block Kit, not plain text:

```
import requests, os  def post_to_slack(tagline, description, pr_url):     payload = {
"channel": "#launches",         "blocks": [             {"type": "header",
"text": {"type": "plain_text", "text": "New Launch: " + tagline}},             {"type":
"section",              "text": {"type": "mrkdwn", "text": description}},
{"type": "section",              "fields": [                  {"type": "mrkdwn", "text":
f"*GitHub PR:* <{pr_url}|View PR>"},                  {"type": "mrkdwn", "text": "*Status:*
Ready for review"}              ]}         ]     }
requests.post("https://slack.com/api/chat.postMessage",         headers={"Authorization":
f"Bearer {os.environ['SLACK_BOT_TOKEN']}"},         json=payload)
```

6.4 Sending Email via SendGrid

Install: pip install sendgrid

```
from sendgrid import SendGridAPIClient from sendgrid.helpers.mail import Mail import os
def send_email(subject, body, to_email):     message = Mail(
from_email="verified@yourdomain.com",         to_emails=to_email,         subject=subject,
html_content=f"<p>{body}</p>"     )     sg =
SendGridAPIClient(os.environ["SENDGRID_API_KEY"])     sg.send(message)
```

6.5 Calling an LLM Inside an Agent

Example using the Anthropic Claude API (works similarly for OpenAI):

```
import anthropic, os  client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])
def call_llm(system_prompt, user_prompt):     response = client.messages.create(
model="claude-sonnet-4-20250514",         max_tokens=1000,         system=system_prompt,
messages=[{"role": "user", "content": user_prompt}]     )     return
response.content[0].text
```

7. What You Must Submit

There are three deliverables. No report. No reflection. Just code, a demo, and a README.

Deliverable 1: Public GitHub Repository

Your repository must contain everything needed to understand and run your system. It must include:

Repository structure:

- README.md see requirements below
- agents/ a folder containing one Python (or JavaScript) file per agent, clearly named (ceo_agent.py, product_agent.py, engineer_agent.py, marketing_agent.py, qa_agent.py)
- main.py (or index.js) a single entry point that runs the entire system end-to-end
- message_bus.py (or equivalent) the shared messaging implementation
- requirements.txt (or package.json) all dependencies
- .env.example a template showing which environment variables are needed, with placeholder values (never real keys)
- .gitignore must include .env so real API keys are never committed

README.md must include:

- What your startup idea is (2-3 sentences)
- A diagram or description of your agent architecture which agent talks to which
- Setup instructions: how to clone the repo, install dependencies, set environment variables, and run the system
- A list of every platform your agents interact with and what they do on each
- A link to the Slack workspace (share invite link) or screenshots of the bot in action
- A link to the GitHub PR created by the Engineer agent

Deliverable 2: Demo Video (8-10 minutes)

Record a screen recording of your entire system running live. The video must show:

25. You running main.py (or equivalent) the terminal must be visible
26. The CEO agent receiving the startup idea and sending messages to sub-agents print these to the terminal as they happen
27. The Product agent generating and outputting its product spec
28. The Engineer agent committing code and opening a PR show the GitHub repository page refreshing with the new PR
29. The Marketing agent sending the email show the inbox with the received email
30. The Marketing agent posting to Slack show the Slack workspace with the bot's message visible
31. At least one feedback loop show the CEO agent receiving a response, reasoning about it, and sending a follow-up instruction
32. The final Slack summary message from the CEO agent

> *Do not edit the video. Do not pre-run the system and play back logs. The evaluator must see the system actually running in real time. If something fails during recording, leave it in show how your system handles failure and explain what happened.*

Deliverable 3: Submission

Submit the following via the course portal:

- The URL of your public GitHub repository
- The URL of your demo video (upload to YouTube, Google Drive, or any accessible link)
- The names of all group members and which agent each member built

---

8. Grading

Criterion	Weight	What the evaluator looks for
Real platform integration	25%	GitHub PR exists and was opened by the agent. Slack message is visible in the workspace. Email arrived in the inbox. These are binary they either happened or they did not.
Agent collaboration and dynamic decision-making	25%	At least one feedback loop where the CEO reasons about another agent's output and sends a revision request. Agents pass structured JSON messages. The message log shows a real conversation, not a fixed sequence.
LLM reasoning quality inside agents	20%	Each agent's LLM prompts are specific and well-engineered. The outputs are coherent and relevant to the startup idea. The CEO's review reasoning is meaningful not just 'looks good'.

Criterion	Weight	What the evaluator looks for
Code quality and architecture	15%	Code is clean and readable. Each agent is in its own file. Environment variables are used for all secrets. The message bus is clearly implemented. The system can be run by a stranger following the README.
Demo and repository	15%	Video shows the full pipeline running live. README is complete. Repository structure is clean. All deliverables are submitted.

Bonus Marks (up to 10% extra)

Bonus criterion	Extra marks
QA agent implemented (for 2-person groups)	+5%
Agents use Redis pub/sub for messaging instead of shared dict	+3%
System handles at least one agent failure gracefully (e.g. API timeout, LLM refusal) and retries or reports the failure to the CEO	+3%
CEO agent runs more than one feedback loop (multiple revision cycles)	+2%
Different agents use different LLM providers (e.g. Claude for CEO, GPT-4o for Engineer)	+2%

9. Common Mistakes Read This Before You Start

Things that will cost you marks:

Hardcoding API keys in your code or committing them to GitHub.

Agents that just call each other in a fixed sequence with no feedback this is not a MAS, it is a pipeline.

GitHub PR not visible in the repository the evaluator will check the URL in your README.

Slack message posted but the bot was removed from the channel before the demo keep the bot in the channel.

LLM outputs that are completely generic and unrelated to your startup idea this means your prompts are not including enough context.

A demo video that is pre-recorded logs being played back must be live.

Not using structured JSON for agent messages string passing is not acceptable.

Frequently Asked Questions

Can we use a mock instead of a real API?

No. GitHub, Slack, and email must be real. There is no partial credit for 'we would have connected it to the real API but did not'. The entire point of this assignment is real-world agent action.

What if our startup idea changes halfway through?

That is fine. Update the README and make sure the final demo reflects the final idea. The idea itself is not graded what matters is what the agents do with it.

Can we use a framework other than the ones listed?

Yes. Any agent framework is acceptable as long as the requirements are met. If you want to build from scratch using direct API calls, that is also fine and earns respect.

What if the LLM refuses to generate something?

Handle it in your code. Catch the error, log it, and have the agent either retry with a revised prompt or report the failure to the CEO. A system that crashes when the LLM misbehaves is not a production-ready MAS.

Do all agents have to run simultaneously?

No. They can run sequentially in a single process. What matters is that they communicate via the message bus and that the CEO is doing real reasoning not just calling functions in order.

Can two agents share the same LLM API key?

Yes. You can use the same API key for all agents. There is no requirement that each agent uses a different account or key.

10. Suggested Timeline

Day	Task
Day 1-2	Decide your startup idea. Set up GitHub repo, Slack workspace, SendGrid account. Create all API keys and store them in .env. Test each platform with a simple script before touching agent code.
Day 3-4	Build the message bus. Build the CEO agent's task decomposition logic (no review loop yet just decompose and send). Build the Product agent. Test CEO -> Product communication end to end.
Day 5-7	Build the Engineer agent. Test GitHub issue creation, file commit, and PR opening independently before wiring it into the pipeline. Build the Marketing agent. Test Slack posting and email sending independently.
Day 8-9	Wire all agents together through the CEO. Add the CEO's review step and at least one feedback loop. Run the full pipeline end to end for the first time.

Day	Task
Day 10-11	Debug. Handle failure cases. Clean up code. Write the README. Build or refine the QA agent if your group has three members.
Day 12-13	Record the demo video. Review the submission checklist. Submit.

11. Submission Checklist

Before you submit, go through every item below. If any item is unchecked, your submission is incomplete.

- GitHub repository is public
- Repository contains agents/ folder with one file per agent
- Repository contains main.py or equivalent entry point
- Repository contains .env.example with placeholder keys
- Repository contains .gitignore with .env listed
- README.md contains startup idea, architecture description, setup instructions, and platform links
- Engineer agent has opened a real pull request visible in the repository
- Marketing agent has posted a real message to the Slack workspace
- Marketing agent has sent a real email verify it arrived in the inbox
- At least one CEO feedback loop is implemented and visible in the demo
- All agent messages are structured JSON with the required fields
- Demo video is 8-10 minutes, shows the system running live, shows all platform actions
- Demo video shows the terminal output with agent messages as they are sent and received
- Submitted: GitHub repo URL, demo video URL, group member names and agent assignments

Good luck. Build something real.